

Python Modules and Real-world Mini Projects

Contents

1	Introduction	5
2	What is Module?	5
3	Importing Module	6
4	OTP Generator	6
4.1	Code	6
5	Understanding randint()	6
6	Why OTP Starts from 100000	6
7	Number Guessing Game	7
7.1	Basic Version	7
8	Game Logic	7
9	Why while True?	7
10	Game with Life System	8
11	Dictionary Usage in Game	8
12	Life Reduction	8
13	Card Shuffler	8
13.1	Card Data	8
14	random.choice()	9
15	Random Card Function	9
16	Creating Full Deck	9
17	Nested Loop	9
18	Shuffling Cards	10

19 Distributing Cards	10
20 Birthday Calculator	10
20.1 Modules Used	10
21 Getting User Input	10
22 Converting String to Date	10
23 Understanding strptime()	11
24 Date Format Symbols	11
25 Getting Today's Date	11
26 Calculating Age	11
27 Showing Age	11
28 Citizenship Example	11
29 Time Module	12
30 Measuring Execution Time	12
31 time.time()	12
32 time.sleep()	12
33 Countdown Timer	12
34 OS Module	13
35 File Organizer Project	13
36 Listing Files	13
37 Joining Paths Safely	13
38 Checking Directory	14
39 Getting File Extension	14
40 Removing Dot	14
41 Creating Folder	14
42 Checking Folder Exists	14
43 Moving File	14
44 User-defined Modules	15

45 What is User-defined Module?	15
46 Importing User-defined Module	15
47 Why User-defined Modules are Important	16
48 Packages	16
49 Why Packages Exist	16
50 Real-world Example	17
51 Simple Analogy	17
52 Important Note	17
53 Additional Exercises	17
53.1 Exercise 11: Create Your Own Module	17
53.2 Exercise 12: Student Utility Module	18
53.3 Exercise 13: Create Package	18
53.4 Challenge 11: Mini Utility Package	18
53.5 Challenge 12: Game Package	18
54 Real-world Importance	19
55 Final Thoughts	19
56 Exercises	20
56.1 Exercise 1: Random Dice	20
56.2 Exercise 2: Coin Toss	20
56.3 Exercise 3: Random Password	20
56.4 Exercise 4: Lucky Draw	20
56.5 Exercise 5: Countdown Timer	20
56.6 Exercise 6: Stopwatch	20
56.7 Exercise 7: Age Calculator	20
56.8 Exercise 8: Card Dealer	20
56.9 Exercise 9: Guessing Game	20
56.10Exercise 10: File Counter	20
57 Challenge Questions	21
57.1 Challenge 1: Advanced OTP System	21
57.2 Challenge 2: Advanced Guessing Game	21
57.3 Challenge 3: Blackjack-like Card Game	21
57.4 Challenge 4: Digital Clock	21
57.5 Challenge 5: Pomodoro Timer	21
57.6 Challenge 6: Auto File Organizer	21
57.7 Challenge 7: Birthday Reminder	22
57.8 Challenge 8: Terminal Animation	22
57.9 Challenge 9: Mini Casino Game	22
57.10Challenge 10: Smart Folder Cleaner	22

1 Introduction

Till now we learned:

- Variables
- Conditions
- Loops
- Lists
- Dictionaries
- Functions

Now we start using:

Python Modules

Modules allow Python to do powerful real-world tasks like:

- Generate OTP
- Shuffle cards
- Work with dates
- Measure execution time
- Manage files and folders

This is where programming starts feeling:

“Oh... this can build actual software.”

2 What is Module?

Module is:

A Python file containing useful functions and tools.

Python already provides many built-in modules.

Examples:

- random
- time
- os
- datetime

3 Importing Module

Before using module:

```
import random
```

This tells Python:

“Please give me access to random module tools.”

4 OTP Generator

4.1 Code

```
import random

otp = random.randint(100000, 999999)

print("Your OTP is:", otp)
```

5 Understanding randint()

```
random.randint(start, end)
```

Generates random integer between:

- start
- end

Including both.

Example:

```
random.randint(1, 5)
```

Possible outputs:

```
1
2
3
4
5
```

6 Why OTP Starts from 100000

If we write:

```
random.randint(0, 999999)
```

OTP may become:

```
4521
```

Only 4 digits.

So:

```
100000
```

ensures minimum 6 digits.

7 Number Guessing Game

7.1 Basic Version

```
import random

secret_number = random.randint(1, 100)

while True:

    guess = int(input("Enter your guess: "))

    if guess < secret_number:
        print("Too low!")

    elif guess > secret_number:
        print("Too high!")

    else:
        print("Correct!")
        break
```

8 Game Logic

Program secretly stores random number.

User keeps guessing.

Program gives hints:

- Too low
- Too high

Loop continues until correct answer.

9 Why while True?

```
while True:
```

means:

Infinite loop

Game continues forever until:

```
break
```

runs.

10 Game with Life System

```
life = 5
```

User gets limited attempts.

11 Dictionary Usage in Game

```
life_to_emotion = {  
    5: "Excellent",  
    4: "Happy",  
    3: "Neutral",  
    2: "Sad",  
    1: "Critical"  
}
```

Dictionary maps:

```
life -> emotion
```

12 Life Reduction

```
life -= 1
```

Same as:

```
life = life - 1
```

13 Card Shuffler

13.1 Card Data

```
suits = ["", " ", " ", " ", " ", " ", " "]  
  
ranks = [  
    "A", "2", "3", "4",  
    "5", "6", "7", "8",  
    "9", "10", "J", "Q", "K"  
]
```


14 random.choice()

```
random.choice(list_name)
```

Selects random item from list.

Example:

```
random.choice(ranks)
```

Possible output:

```
Q
```

15 Random Card Function

```
def get_random_card():  
  
    return f"{random.choice(suits)}  
           {random.choice(ranks)}"
```

Possible output:

```
K
```

16 Creating Full Deck

```
all_cards = []  
  
for suit in suits:  
  
    for rank in ranks:  
  
        card = f"{suit} {rank}"  
  
        all_cards.append(card)
```

Nested loop creates:

- 52 cards

17 Nested Loop

Outer loop:

- Suit changes

Inner loop:

- Rank changes

This creates all possible combinations.

18 Shuffling Cards

```
random.shuffle(all_cards)
```

Shuffles list randomly.

Important:

- shuffle changes original list

19 Distributing Cards

```
print(all_cards[0])
print(all_cards[1])
print(all_cards[2])
```

Top cards go to players.

20 Birthday Calculator

20.1 Modules Used

```
from datetime import datetime, date
from dateutil.relativedelta import relativedelta
```

21 Getting User Input

```
dob_input = input(
    "Enter your birthdate (YYYY-MM-DD): "
)
```

Input is string.

Example:

```
"2000-01-01"
```

22 Converting String to Date

```
birth_date = datetime.strptime(
    dob_input,
    "%Y-%m-%d"
).date()
```

23 Understanding `strptime()`

```
strptime()
```

means:

String Parse Time

Converts string into date object.

24 Date Format Symbols

```
%Y
```

4-digit year.

```
%m
```

Month.

```
%d
```

Day.

25 Getting Today's Date

```
today = date.today()
```

26 Calculating Age

```
age = relativedelta(today, birth_date)
```

27 Showing Age

```
print(  
    f"Age: {age.years} years,  
    {age.months} months,  
    {age.days} days"  
)
```

28 Citizenship Example

```
if age.years >= 18:  
    print("Eligible")  
else:  
    print("Not eligible")
```

Real-world logic.

29 Time Module

```
import time
```

Used for:

- Delays
- Timers
- Performance checking

30 Measuring Execution Time

```
start = time.time()

time.sleep(5)

end = time.time()

print(end - start)
```

31 time.time()

Returns:

Current timestamp in seconds.

32 time.sleep()

```
time.sleep(5)
```

Pauses program for:

- 5 seconds

33 Countdown Timer

```
num = 10

while num > 0:

    print(num)

    time.sleep(1)

    num -= 1
```

Program waits:

- 1 second each loop

Creates real countdown effect.

34 OS Module

```
import os
```

OS means:

Operating System

Allows Python to interact with:

- Files
- Folders
- System commands

35 File Organizer Project

Goal:

Automatically organize files into folders.

Example:

```
photo.png -> png/  
song.mp3 -> mp3/  
notes.txt -> txt/
```

36 Listing Files

```
files = os.listdir(folder_path)
```

Returns all files and folders.

37 Joining Paths Safely

```
os.path.join(folder_path, file)
```

Safely builds file path.

Very important because:

- Windows and Linux paths differ

38 Checking Directory

```
os.path.isdir(file_path)
```

Checks whether path is folder.

39 Getting File Extension

```
_, ext = os.path.splitext(file)
```

Example:

```
hello.txt
```

becomes:

```
("hello", ".txt")
```

40 Removing Dot

```
ext = ext[1:]
```

Converts:

```
.txt
```

into:

```
txt
```

41 Creating Folder

```
os.makedirs(target_folder)
```

Creates folder automatically.

42 Checking Folder Exists

```
os.path.exists(target_folder)
```

Prevents duplicate folder creation.

43 Moving File

```
os.rename(old_path, new_path)
```

Moves file into correct folder.

44 User-defined Modules

Till now we used built-in modules like:

- random
- os
- time

But programmers can also create their own modules.
This is called:

User-defined Module

45 What is User-defined Module?

A normal Python file:

```
.py
```

can become module.

Example:

Create file:

```
math_utils.py
```

Inside file:

```
def add(a, b):  
    return a+b  
  
def multiply(a, b):  
    return a*b
```

Now another file can use it.

46 Importing User-defined Module

```
import math_utils  
  
print(math_utils.add(4, 5))
```

Output:

```
9
```

47 Why User-defined Modules are Important

They help:

- Reuse code
- Organize projects
- Reduce duplication
- Separate logic cleanly

Real software projects may contain:

- 50 modules
- 100 modules
- Even 1000+ modules

48 Packages

When many modules are grouped inside folder:

It becomes package.

Example:

```
project/  
  
    maths/  
        add.py  
        subtract.py  
        multiply.py
```

Here:

```
maths
```

is package.

49 Why Packages Exist

Imagine keeping:

- 500 Python files

inside single folder.

Absolute chaos.

Packages help organize modules properly.

50 Real-world Example

Popular Python packages:

- numpy
- pandas
- django
- flask

These packages internally contain:

- Many modules
- Many sub-packages
- Thousands of functions

51 Simple Analogy

- Function = one worker
- Module = one department
- Package = whole office building

52 Important Note

Earlier Python versions required:

```
__init__.py
```

inside package folder.

Modern Python can work without it in many cases.

But many developers still keep it for clarity and compatibility.

53 Additional Exercises

53.1 Exercise 11: Create Your Own Module

Create file:

```
calculator.py
```

Add functions:

- add()
- subtract()
- multiply()
- divide()

Import and use them from another file.

53.2 Exercise 12: Student Utility Module

Create module:

```
student_utils.py
```

Functions:

- `calculategrade()`
- `calculatepercentage()`

53.3 Exercise 13: Create Package

Create package:

```
bank/
```

Modules:

- `deposit.py`
- `withdraw.py`
- `balance.py`

Import and test functions.

53.4 Challenge 11: Mini Utility Package

Create package:

```
utils/
```

Modules:

- `mathutils.py`
- `stringutils.py`
- `fileutils.py`

Each should contain useful helper functions.

53.5 Challenge 12: Game Package

Create package:

```
games/
```

Modules:

- `dicegame.py`
- `guessinggame.py`
- `cardgame.py`

Create main program importing and running games.

54 Real-world Importance

These concepts are heavily used in:

- Automation
- Cybersecurity
- DevOps
- Data Science
- Backend development
- System scripting

This is where Python becomes:

A real productivity superpower.

55 Final Thoughts

Today you learned:

- Modules
- random module
- time module
- os module
- datetime module
- OTP generation
- Games
- Timers
- File handling basics

You are now entering:

Real-world Python programming.

56 Exercises

56.1 Exercise 1: Random Dice

Generate random dice number between:

- 1 to 6

56.2 Exercise 2: Coin Toss

Randomly print:

- Head
- Tail

56.3 Exercise 3: Random Password

Generate random 8-character password.

56.4 Exercise 4: Lucky Draw

Store 5 names in list.

Select random winner.

56.5 Exercise 5: Countdown Timer

Create countdown from:

- 10 to 1

56.6 Exercise 6: Stopwatch

Measure how long user takes to press Enter twice.

56.7 Exercise 7: Age Calculator

Take DOB input.

Print current age.

56.8 Exercise 8: Card Dealer

Generate 5 random cards.

56.9 Exercise 9: Guessing Game

Allow only:

- 3 attempts

56.10 Exercise 10: File Counter

Count total files inside folder.

57 Challenge Questions

57.1 Challenge 1: Advanced OTP System

Generate OTP with:

- Numbers
- Letters
- Expiry timer

57.2 Challenge 2: Advanced Guessing Game

Add:

- Difficulty levels
- Score system
- Replay option

57.3 Challenge 3: Blackjack-like Card Game

Create simplified card game using shuffled deck.

57.4 Challenge 4: Digital Clock

Create real-time digital clock using:

```
while True
```

and:

```
time.sleep()
```

57.5 Challenge 5: Pomodoro Timer

Create study timer:

- 25 min study
- 5 min break

57.6 Challenge 6: Auto File Organizer

Improve organizer:

- Images folder
- Videos folder
- Documents folder

57.7 Challenge 7: Birthday Reminder

Store birthdays and show upcoming birthdays.

57.8 Challenge 8: Terminal Animation

Create loading animation using:

- `time.sleep()`
- loops

57.9 Challenge 9: Mini Casino Game

Create:

- Dice game
- Coin toss game
- Betting system

57.10 Challenge 10: Smart Folder Cleaner

Automatically:

- Delete empty folders
- Rename duplicate files
- Organize by extension

58 Homework Advice

The projects you built today may look small.

But these same concepts are used in:

- Real automation tools
- Games
- Security systems
- File management software
- Enterprise applications

Professional developers simply combine:

Small concepts + logic + creativity

to build powerful systems.

That journey has already started.